.NET Core Interview Questions and Answers

1. What is .NET Core, and how is it different from the .NET Framework?

.NET Core is a cross-platform, high-performance framework for building modern cloud-based, internet-connected applications. Unlike the .NET Framework, which runs only on Windows, .NET Core supports Windows, macOS, and Linux, making it more versatile for cross-platform development.

2. How do you install .NET Core SDK?

You can download the .NET Core SDK from the official .NET website. Follow the installation guide for your specific operating system (Windows, macOS, or Linux).

3. What is the purpose of the dotnet new command?

The dotnet new command creates a new .NET Core project with a specific template (e.g., console app, web app, etc.). For example, dotnet new console creates a new console application.

4. How does .NET Core support cross-platform development?

.NET Core is designed to be platform-independent. It uses a runtime (CoreCLR) that can run on different operating systems. The same .NET Core application can run on Windows, macOS, and Linux without modification.

5. What are the main benefits of using .NET Core for application development?

a. Cross-platform support (Windows, macOS, Linux)
b. High performance and scalability
c. Modern development (support for cloud, microservices)
d. Open-source and community-driven
e. Unified development (build web, desktop, mobile, and cloud apps)

6. What is the role of the Common Language Runtime (CLR) in .NET Core?

The CLR is the runtime environment in .NET Core that manages the execution of .NET applications, providing services like memory management, exception handling, and garbage collection.

7. Explain the structure of a .NET Core project.

A typical .NET Core project contains files like Program.cs (entry point), Startup.cs (configuration), and appsettings.json (configuration settings). It also includes directories like Controllers, Views, wwwroot, and dependencies listed in the csproj file.

8. What is the difference between dotnet run and dotnet build commands?

a. dotnet build compiles the application without running it.

b. dotnet run compiles and runs the application.

9. How do you create a new ASP.NET Core MVC project from scratch?

Run the command **dotnet new mvc -n ProjectName** to create a new ASP.NET Core MVC project. Then, navigate to the project directory and run dotnet run to launch the project.

10. Authentication Vs Authorization in .NET Core?

Authentication verifies the identity of a user, whereas authorization determines whether that authenticated user has permission to access a particular resource. In ASP.NET Core, authentication is handled by UseAuthentication() and authorization by UseAuthorization(), in that order.

11. What is Dependency Injection in .NET Core ?

Dependency Injection is an Design Pattern used to achieve loosely coupled in application. Instead of creating dependencies (object or services) directly within the class, dependencies are injected externally outside from the class, typically through constructor pattern. Also, we can register these services in the Program.cs file through service lifetimes such as AddSingleton, AddScoped, and AddTransient.

12. How does .NET Core support dependency injection?

.NET Core has built-in support for dependency injection (DI) through the IServiceCollection and IServiceProvider interfaces. You can register services in the Program.cs file using methods like AddSingleton, AddScoped, and AddTransient.

13. What is Middleware is in ASP.NET Core. How does it work?

Middleware in ASP.NET Core is a component that is part of the HTTP request pipeline. It processes incoming HTTP requests and outgoing HTTP responses.

Each middleware component can:

1. Process the request
2. Pass the request to the next middleware
3. Short-circuit the pipeline by generating a response without calling the next middleware.

How it works

Middleware components are executed in the same order in which they are registered in Program.cs

Common Middleware components:

- **UseRouting**: Sets up endpoint routing.
- **UseAuthentication**: Handlesauthentication.
- **UseAuthorization**: Handles authorization based on user roles or claims.
- **UseStaticFiles**: Serves static files like CSS, JavaScript, and images.
- **UseExceptionHandler**: Handles exceptions globally.

14. Difference between app.Use( ) and app.Run( ) ?

- app.Use() adds middleware that can execute code before and after the next middleware, whereas app.Run() adds terminal middleware that ends the request pipeline and does not call the next middleware

15. Diffrence between Map( ) and MapGet( ) ?

Map branches the middleware pipeline by path prefix and is not tied to any HTTP verb, while MapGet registers a GET endpoint for a specific route template in Minimal APIs. In short, Map creates a separate pipeline for a sub-path, whereas MapGet handles a specific GET request and returns a response directly.

16. AddController vs AddMvc vs AddControllersWithViews vs AddRazorPages ?

Method	Controller (API)	Views	Razor Pages	Use Case
AddController()	✅	❌	❌	Web API only
AddMvc()	✅	✅	❌	MVC app (controllers + Razor views)
AddControllersWithViews()	❌	❌	✅	Razor Pages app
AddRazorPages()	✅	✅	✅	Everything (legacy, heavier)

17. How does routing work in ASP.NET Core MVC?

Routing in ASP.NET Core MVC maps incoming requests to controller actions. Routes are defined in the Startup.cs file, typically using the UseEndpoints or MapControllerRoute methods.

18.Difference between Attribute and Conventional routing

In attribute routing, routes are defined directly on controllers and action methods using attributes such as [Route], [HttpGet], [HttpPost], etc.
Conventional routing defines a common route pattern centrally, such as {controller}/{action}/{id?}, and ASP.NET Core uses that convention to map requests to controllers and actions.

19. What is an API controller, and how does it differ from an MVC controller in .NET Core?

API controllers are specifically used for building Web APIs. They return data (e.g., JSON) rather than views. API controllers use the [ApiController] attribute to enable features like automatic model validation.

20. What is appsettings.json, how do you configure settings in it?

appsettings.json is a configuration file used in .NET Core applications to store settings like database connection strings, environment settings, and more. You can access these settings through the IConfiguration interface in .NET Core.

21. What is Entity Framework Core, and what is its primary use?

Entity Framework Core (EF Core) is an Object-Relational Mapping (ORM) framework that allows developers to interact with databases using .NET objects, eliminating the need for most SQL queries.

22. How do you connect a .NET Core application to a SQL Server database?

Add a connection string to appsettings.json, then configure it in Program.cs using the AddDbContext method. You can then inject the DbContext into your application to interact with the database.

23. How does the IConfiguration interface work in .NET Core?

The IConfiguration interface provides access to configuration settings, allowing you to read key-value pairs from files like appsettings.json, environment variables, or command-line arguments.

24. What are NuGet packages, and how are they used in .NET Core projects?

NuGet is a package manager for .NET. NuGet packages are libraries that you can add to your project to use third-party or shared code. You can install them via the dotnet add package command.

25. Explain how you handle errors using try-catch in .NET Core.

The try-catch block is used to handle exceptions. You place the code that might throw an exception inside the try block and handle the exception in the catch block.

26. What is the role of the Program.cs file in a .NET Core project?

Program.cs is the entry point of a .NET application. It contains the code that starts the application, configures the host, registers services, and defines the HTTP request pipeline.

27. How do you enable HTTPS in a .NET Core application?

To enable HTTPS, configure your project to use HTTPS in launchSettings.json. Also, add middleware like UseHttpsRedirection in the Startup.cs file.

28. How do you implement basic logging in .NET Core?

.NET Core provides built-in logging support through the ILogger interface. You can inject ILogger into controllers or services and log information using methods like LogInformation, LogWarning, and LogError.

29. What is model binding in ASP.NET Core MVC?

Model binding is the process of mapping incoming request data (such as form data or query parameters) to action method parameters.

30. Explain what ViewData, ViewBag, and TempData are in ASP.NET Core.

a. ViewData: A dictionary for passing data from controller to view.

b. ViewBag: A dynamic wrapper around ViewData for easier access to data.

c. TempData: Used to store data temporarily between two requests, typically after a redirect.

31. How do you implement validation in ASP.NET Core using data annotations?

Data annotations are attributes applied to model properties to enforce validation. For example, [Required], [Range], and [StringLength] are used for validation. These attributes are checked automatically by the model binder.

32. What is a RESTful API, and how do you build one using ASP.NET Core?

A RESTful API follows REST principles like stateless communication, resource representation, and HTTP methods. You can create a RESTful API in ASP.NET Core by creating controllers that return data (usually JSON) and use HTTP methods like GET, POST, PUT, and DELETE.

33. How do you consume a Web API in a .NET Core application?

You can use HttpClient to consume a Web API. Create an instance of HttpClient, send a request using GetAsync or PostAsync, and process the response.

34. How do you manage cookies in an ASP.NET Core application?

You can manage cookies using the HttpContext.Response.Cookies.Append method to set cookies and HttpContext.Request.Cookies to read them.

35. What are filters in ASP.NET Core?

Filters allow you to run code before or after an action method executes. Examples include authorization filters, resource filters, action filters, and exception filters.

36. How do you handle user authentication in an ASP.NET Core application?

ASP.NET Core supports authentication through Identity, JWT, or OAuth. You

configure authentication in Startup.cs and use middleware to handle login, registration, and token validation.

37. What is Razor, and how is it used in ASP.NET Core MVC?

Razor is a templating engine used to generate dynamic HTML in MVC views. It allows you to embed C# code into HTML using the @ symbol.

38. How do you serve static files in a .NET Core application?

Use the Use Static Files middleware in Startup. cs to serve static files from the www root folder.

39. What is the View Component in ASP.NET Core MVC?

A View Component is a reusable component that renders part of a page. It is similar to partial views but allows for more complex logic.

40. How do you return JSON data from a controller in ASP.NET Core?

Return a JSON result from an action method using return J son(object) or return Ok(object) in an API controller.

41. How do you perform file uploads in ASP.NET Core?

Use the I Form File interface to handle file uploads. In the controller, read the uploaded file, process it, and save it to a specified location.

42. What are Tag Helpers in ASP.NET Core?

Tag Helpers are server-side components that generate HTML and provide a way to interact with Razor views. Examples include asp-for, asp-controller, and asp-action.

43. How do you publish a .NET Core application?

Use the dotnet publish command to compile and package the application for

deployment. You can also specify the target runtime (e.g., Windows, Linux) during the publishing process.

44. What are global exception handlers, and how do you implement them in .NET Core?

Global exception handling can be implemented using middleware. You create custom middleware that catches exceptions and handles them centrally, ensuring that all unhandled exceptions are captured.

45. Explain the difference between IActionResult and ActionResult in an ASP.NET Core controller.

Both IActionResult and ActionResult are used as return types for ASP.NET Core controller actions, but there are some key differences:

IActionResult:

1. IActionResult is an **interface** that represents a wide variety of possible return types for controller actions.
2. It allows for greater flexibility as it can represent any type that implements IActionResult, such as ViewResult, JsonResult, RedirectResult, ContentResult, etc.

46. What is model validation, and how do you handle it in ASP.NET Core MVC?

Model validation is the process of ensuring that incoming data from HTTP requests conforms to the rules defined in your data models. ASP.NET Core provides a built-in mechanism to validate models using **data annotations**.

Defining validation rules: Use data annotations to apply validation rules to your model properties:

47. What are Filters in ASP.NET Core MVC? What are the types of filters?

Filters in ASP.NET Core MVC are used to execute custom logic before or after certain stages in the request pipeline, such as before an action is executed or after a result is returned. Filters provide a way to implement cross-cutting concerns like logging, authorization, caching, and exception handling.There are several types of filters:

Authorization Filters:

1. Run before the action is executed to determine whether the user is authorized to execute the action.
2. Example: [Authorize] attribute is an authorization filter.

48. What are Tag Helpers in ASP.NET Core?

Tag Helpers in ASP.NET Core are used to extend the functionality of HTML elements in Razor views. They allow you to add server-side processing logic to HTML tags in a clean and readable way, making your Razor views more expressive and maintainable.Tag Helpers are processed on the server, and they help you build dynamic content while still working with familiar HTML syntax.

49. How does ASP.NET Core support Cross-Origin Resource Sharing (CORS)?

Cross-Origin Resource Sharing (CORS) is a security feature implemented by web browsers to prevent unauthorized access to resources from different origins (domains). ASP.NET Core provides built-in support for configuring CORS policies to allow or restrict cross-origin requests.

a. **Enable CORS in Startup.cs**: You need to configure CORS in the Startup.cs file by using the services.AddCors() method inside ConfigureServices and applying CORS policies in Configure().

50. What is View Model in ASP.NET Core MVC? How is it different from a Model?

In ASP.NET Core MVC, a **View Model** is a class that represents the data and logic required specifically for the view. It is designed to transfer data between the controller and the view, and is often a combination of multiple models or properties tailored for the view's needs. **Difference between View Model and Model:**

Model:

a. Represents the domain or business logic of the application.

b. Contains data and validation rules for entities like User, Product, etc.

c. Used to interact with the database through ORM frameworks like Entity Framework.

ViewModel:

1. A custom class designed for the view, containing only the data needed for display or user interaction.
2. Can include properties from multiple models, as well as additional logic or formatting specific to the view.

3. Not tied to the database.

51. How do you implement Authentication and Authorization in ASP.NET Core?

Authentication and **Authorization** are essential aspects of securing ASP.NET Core applications.

a. **Authentication**:

   i. Authentication is the process of verifying the user's identity (e.g., login credentials).

   ii. ASP.NET Core supports various authentication mechanisms such as **cookies**, **JWT tokens**, **OAuth**, and **OpenID Connect**.

b. **Authorization**:

   i. Authorization determines what actions a user can perform after their identityhas been authenticated.

   ii. ASP.NETCoreuses **roles** or **claims** forimplementingauthorization.

53. What is ViewComponent in ASP.NETCore, and how is it different froma Partial View?

A **ViewComponent** in ASP.NETCore is areusable componentthatencapsulates renderinglogic, similar to acontrolleractionbutwiththepurpose of renderingaspecific part of aview. ViewComponentsareideal for buildingcomplex UI componentsthatmay includetheirown logicandmodels, independent ofthemainpage or view model.

Keyfeatures of ViewComponent:

1. It is **reusable** acrossmultipleviews.
2. It hasitsown **logic** and **view**.
3. It doesn't participate in the MVCaction pipeline but is invoked directly from a view.
4. It canreturn **HTML**, **JSON**, or otherformats.

Differences between ViewComponent and PartialView:

- **ViewComponent**: Contains both rendering logic(C# code)andaview(UIpart). It has its ownmodelandcan perform complex logic before renderingtheview.
- **PartialView**: Is purely a view (UIcomponent) without any logic. It sharesthemodel of the parent view.

54. HowdoessessionmanagementworkinASP.NETCore?

In ASP.NET Core, **session management** is used to store **temporary data** specific to a user across multiple HTTP requests. Unlike cookies, session data is stored server-side, and a **session ID** is passed to theclientviaacookie.

Howsessionmanagementworks:

1. Session data is stored in the server's memory, a distributed cache, or a database.
2. The session ID is stored in acookie on the client.
3. Each time the usermakes a request, the session ID is sent with the request, and the server retrieves the corresponding session data.

55. Whatis UseEndpoints() in ASP.NETCore,andhow does it differfrom UseMvc()?

UseEndpoints() and UseMvc() are methods in ASP.NET Core used to configure request handling, but they differ in how they manage routing and endpoints.

UseMvc():

1. Part of the **olderroutingsystem** in ASP.NETCore 2.x.
2. Configures routingbased on controlleractions, withroutesdefinedexplicitlyin Startup.cs or using attribute-based routing.
3. It wascommonlyusedwith **services.AddMvc()** in the ConfigureServices() method.

56 What is the difference between a public API returning Ok(user) and returning a DTO? Why can return Ok(user) be dangerous?

Returning the entity directly can expose internal or sensitive properties, create over-posting/data-contract problems, and tightly couple the API contract to the database model. Prefer a DTO that contains only the fields the client is allowed to receive.

57 How would you implement global exception handling in ASP.NET Core?

Use centralized exception-handling middleware, such as UseExceptionHandler or custom middleware. Catch unhandled exceptions, log the exception, map known exceptions to appropriate HTTP status codes, and return a consistent error response without exposing internal details.

58 What is the lifecycle of a .NET Core / ASP.NET Core application?

The application starts by building configuration and the dependency-injection container, creates the host, registers and builds the request pipeline, starts the web server such as Kestrel, and then processes requests through the middleware and endpoint pipeline. On shutdown, hosted services are stopped and the host is disposed.

59 What happens internally when you register a service as Singleton, Scoped, or Transient? When can each be a bad choice?

Singleton creates one instance for the application's lifetime, Scoped normally creates one instance per request scope, and Transient creates a new instance each time it is resolved. Singleton is a bad choice for mutable request-specific state; Scoped is unsuitable when a dependency must live beyond the request; Transient can be inefficient for expensive, stateless services created very frequently.

60 What happens if a Singleton depends on a Scoped service?

It creates a lifetime mismatch because the Singleton outlives the Scoped dependency. ASP.NET Core normally detects this when scope validation is enabled and throws an error. A Singleton should not directly depend on a Scoped service; instead, create a scope when appropriate or redesign the dependency.

61 Why is DbContext usually registered as Scoped in ASP.NET Core?

DbContext is designed around a unit-of-work/request scope. A Scoped lifetime gives each HTTP request its own context and change tracker, avoids sharing mutable tracking state between concurrent requests, and allows all operations in one request to participate in the same unit of work.

62. If a .NET Web API suddenly becomes slow in production, where would you start troubleshooting?

Start by measuring rather than guessing: check request latency, error rate, CPU, memory, database duration, external calls, logs, and distributed traces. Identify whether the bottleneck is application code, database queries, network dependencies, thread-pool starvation, or infrastructure, then reproduce/profile the specific bottleneck and fix the highest-impact cause.

63 How would you handle high-volume concurrent requests in an ASP.NET Core Web API?

Use asynchronous I/O, avoid blocking calls, keep request handlers lightweight, use connection pooling and efficient database access, cache suitable read-heavy data, and scale out when necessary. Also apply rate limiting, timeouts, cancellation tokens, and appropriate connection/resource limits so traffic cannot exhaust dependencies.

64 How do you implement idempotency in a payment-processing API?

Require a unique idempotency key for each logical payment request. Store the key with the operation/result in a durable store under a unique constraint, and return the previously stored result when the same key is retried. This prevents duplicate processing when clients retry after timeouts.

65 How would you handle a background job that processes the same message multiple times?

Design the consumer to be idempotent. Persist a unique message/event identifier or business operation key and reject or safely ignore duplicates. Combine this with appropriate queue acknowledgment, retry, dead-letter, and transactional/outbox patterns where required.

66 What is the Unit of Work Pattern in a .NET application?

A Unit of Work coordinates multiple changes and commits them as one logical operation. EF Core's DbContext already provides Unit-of-Work behavior through change tracking and SaveChanges(), so an additional UnitOfWork wrapper is useful only when the application has a clear architectural reason for it.

67 What is CORS in ASP.NET Core and how do you configure it?

CORS controls whether a browser is allowed to make cross-origin requests to the API. Configure named or policy-based CORS rules with AddCors and apply the appropriate policy in the request pipeline. Allow only required origins, methods, and headers rather than using an unrestricted policy in production.

68 Explain REST APIs, HTTP methods, and common HTTP status codes in ASP.NET Core.

A REST API exposes resources through HTTP and uses methods according to their semantics: GET for retrieval, POST for creation/commands, PUT for replacement, PATCH for partial updates, and DELETE for removal. Common responses include 200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, and 500 Internal Server Error.